

PII Redaction Tool

PII Redaction Tool - approach, scope decisions and trade-offs | Arun

A script that reads a .docx and writes a copy with the personal information swapped out for believable fake values. I built and tested it against the Red Herring Prospectus that came with the assignment (KSH International Limited, 519 pages).

```
pip install -r requirements.txt

python redact.py "Red Herring Prospectus.docx" \
  -o "Red Herring Prospectus (redacted).docx" \
  --mapping mapping.json \
  --audit audit.jsonl
```

Takes about 20 seconds on the full document. My output is in out/.

How it works

I ended up with two quite different strategies, because the PII in this document splits into two groups.

Things with a fixed shape

Emails, phone numbers, SSNs, card numbers, IPs, Aadhaar, PAN, CIN, DIN and SEBI registration numbers all follow a spec. For these a regex plus a validity check beats anything cleverer:

- card numbers have to pass Luhn and start with a real network prefix
- Aadhaar has to pass the Verhoeff checksum
- SSNs can't use the ranges the SSA never issues (000, 666, 9xx area, 00 group, 0000 serial)

Phone numbers were the fiddly one. My first version matched any run of 8-15 digits and immediately started eating share counts and rupee figures, of which a prospectus has thousands. So I tightened it: a number only counts if it has an explicit +country prefix, or if it follows a label like Telephone: or Fax: . Bare digits are never a phone number.

Dates were the same problem, worse. This document is nothing but dates - board resolutions, fiscal year ends, consent letters, every single one formatted identically to a date of birth. Redacting them all would have destroyed the document. So a date only counts as a DOB if something like "date of birth", "born on" or "DOB" appears within 60 characters of it.

Names, companies and addresses

These have no fixed shape, so I do a learning pass over the whole document before redacting anything.

The first pass looks for places where the document *tells* you something is a name. "Contact Person:" lines. Honorifics. "X is our Company Secretary". "X, Managing Director". The "OUR PROMOTERS:" list on the cover. And best of all, any table that has both a Name column and a DIN or Designation column, which is always a directors or KMP table. Companies come from legal suffixes (Private Limited, LLP, Corporation, Family Trust). Addresses come from a PIN code with address vocabulary near it, or an "Address:" label.

The second pass spreads those findings across the document, and this is where most of the recall actually comes from:

- every token of a confirmed name joins a vocabulary, so once "Kushal Subbayya Hegde" is known, "Mr. Hegde" and "Kushal Hegde" get caught too

- two rounds of expansion pick up capitalised phrases sharing a token with a known name. This is how "Karunakar Hegde" gets found - he appears once, in a share transfer log, and nothing else points at him. The round after that finds "Karunakar N. Bhandary"
- company short forms propagate, so "KSH International Limited" also covers "KSH International"
- address fragments propagate too. Addresses here are multi-line blocks where each line is its own paragraph, so only the line with the PIN code is findable on its own. Harvesting pieces like "Village Birdewadi" from the addresses I *can* anchor catches the orphaned lines

The optional NER layer

`optional_ner_layer()` folds spaCy PERSON and ORG entities into the same seed sets. It's on by default and falls back to the rules if spaCy or the model isn't installed, so a missing dependency never breaks the run.

The numbers in EVALUATION.md were produced **without** it - I had no network route to the spaCy model host while building this. To reproduce with NER: `python -m spacy download en_core_web_lg && python redact.py . . .` It should mainly help with the one thing my rules can't reach, which is companies with no legal suffix.

Picking the replacements

Every real value maps to exactly one fake value, so the document still makes sense afterwards. The same director keeps the same alias in the shareholding table, the signature block and the risk factors, and their surname on its own resolves to that alias's surname.

Fakes come from Faker (en_IN) seeded with a SHA-256 of the real value, which means two runs produce identical output and there's no mapping file to carry around. Shapes are preserved where it matters: phone numbers keep their country code and punctuation, companies keep their legal suffix, card numbers stay Luhn-valid. Email local parts built from someone's name follow that person's alias, and domains follow the company's alias, so an email and a website for the same company stay consistent with each other.

One thing that caught me out: Faker's Indian locale draws from the same name pool as the document, so it cheerfully invented "Vijay Hegde" while I was busy redacting "Rajesh Hegde". Harmless, but it reads like a leak and makes the output impossible to spot-check. The surrogate bank now refuses to emit anything containing a real token from the source.

Writing back to the .docx

Detection runs on the full paragraph text, but replacement is written back run by run. Word splits runs at every formatting change, often mid-word and often mid-email-address, so a per-run regex misses most matches while a whole-paragraph rewrite throws away the formatting. Runs with no PII in them aren't touched at all.

Three places I nearly missed:

1. **Field codes.** HYPERLINK "mailto:cs.connect@kshinternational.com" isn't visible text - it lives in `w:instrText`. There are 117 of them in this document. I only found it by unzipping my own output and grepping the XML, at which point 12 real addresses were still sitting there while the visible text showed fakes. This was the single biggest bug I hit.
2. **Headers, footers and hyperlink relationship targets.**
3. **Table cells.** I originally walked cells via `row.cells` and de-duplicated merged cells by `id()`. That was quietly wrong: lxml proxy objects are temporary, and a freed proxy's `id()` gets reused, so cells were being treated as already-seen. It skipped 2,611 of 5,041 paragraphs and I didn't notice until I compared paragraph counts. Now I walk the `w:tc` elements directly.

What I decided counts as PII

The brief asked me to be explicit about this, so:

Redacted - person names including partial mentions and historical shareholders; commercial entities (issuer, lead managers, bankers, auditors, lawyers, registrars, group companies, promoter trusts, suppliers, customers, peers); postal addresses including directors' home addresses; emails, phones and corporate websites; and CIN, DIN and SEBI registration numbers, which pin down a specific company or director just as precisely as a name does.

Left alone

- **Regulators and market infrastructure** - SEBI, RBI, RoC, NSE, BSE, NSDL, CDSL, NPCI, government ministries and their websites. They identify no private party, the document is unreadable without them, and redacting them tells the reader nothing.
- **Statutes, defined terms and jargon** - "Companies Act, 2013", "Anchor Investor", "Escrow Collection Bank". Capitalised and entity-shaped, but not personal information.
- **Bare place names** - "Pune", "Maharashtra", "Birdewadi" standing alone. Only full postal addresses get redacted. A city name on its own identifies nobody.
- **Non-birth dates, money, share counts, page references.** This is the main precision trade-off and it's a deliberate one.
- **Order and ticket numbers**, per the brief's example. The negative controls in the test suite check these survive.

What it misses, and what nearly went wrong

Misses

The real gap is **companies with no legal suffix**. Financial Express, Jansatta and Loksatta (newspapers), Sterlite Copper (a brand), and KSH International Chakan Internal Kamgar Sangathna (a trade union) all slip through, because the whole ORG detector is anchored on incorporation markers. Every remaining false negative in my evaluation is one of these. Turning on the spaCy layer is the intended fix; a hand-written seed list would be the deterministic alternative.

Smaller ones: Chakan Unit No. 2 (Birdewadi) keeps the village name, which follows from my scope rule but is arguably wrong given the unit number next to it. And someone whose name shares no token with any other name, appearing in no structural cue, would be missed entirely - expansion needs something to latch onto.

Precision problems I had to fix

Measured precision is 1.000, but it absolutely wasn't at first. Things that fired during development:

- Promoters, Name and Promoter Group detected as people, because my directors-table harvester was feeding header and label cells straight in without any shape check
- the promoter trusts detected as *people* rather than companies, off the cover page list
- India redacted 84 times as a company name, along with State, Refund Bank, Escrow Collection Bank and Practicing Company. Fixed by splitting legal suffixes into strong and weak, guarding the weak ones against offer-machinery vocabulary, and refusing single-token short forms entirely
- Nuvama Wealth Management Limited and ICICI Securities Limited captured as one company, so they shared an alias. Now split on conjunctions
- Yojana PV Photo Voltaic PWIL Precision Wires India Limited, which is two adjacent table cells glued together. Company names can no longer span a newline
- Shanti Gopalkrishnan Sebi Registration and Indu Jacob Independent - field labels and role words getting absorbed into names during expansion

Still rough

- Address boundaries are approximate. A few spans swallow a leading label, e.g. Registered Office: 11/3, ... Over-redaction rather than under, but untidy.
- A field code split across several `instrText` nodes would be handled per node. Didn't happen in this document, but it could.
- The DIN rule treats any bare zero-leading 8-digit number as a DIN. Right here, wrong on a document that uses zero-padded reference numbers.
- PII inside an embedded image survives. That needs OCR.

Adding a new PII type

Two places, which was the point of structuring it this way:

```
# 1. pii_redactor/detectors.py - how to find it
@register
class PassportDetector(Detector):
    pii_type = "PASSPORT"
    name = "passport.regex"
    _RE = re.compile(r"\b[A-PR-WY][1-9]\d\s?\d{4}[1-9]\b")

    def detect(self, text, context):
        for m in self._RE.finditer(text):
            yield self._span(m.start(), m.end(), text)

# 2. pii_redactor/surrogates.py - how to replace it
def passport(self, mention: str) -> str: ...
_DISPATCH = {..., "PASSPORT": "passport"}
```

Then add a priority in `core.PRIORITY` so overlap resolution knows who wins. Overlap handling, the docx rewriting, the audit log and the evaluation harness are all type-agnostic and don't change. Set `needs_fit = True` if the detector wants a document-level learning pass first.

Files

Path	What it is
<code>redact.py</code>	CLI
<code>evaluate.py</code>	Scorer, plus the synthetic test suite
<code>pii_redactor/core.py</code>	Span type, detector base class, registry, overlap resolution
<code>pii_redactor/detectors.py</code>	One class per PII type
<code>pii_redactor/surrogates.py</code>	Fake value generation
<code>pii_redactor/lexicons.py</code>	All the word lists in one place
<code>pii_redactor/docx_io.py</code>	docx traversal and run-level rewriting
<code>pii_redactor/pipeline.py</code>	Ties it together
<code>eval/gold.json</code>	My hand-annotated gold standard, 140 paragraphs
<code>eval/sample.json</code>	The sampled paragraphs
<code>eval/results.json</code>	Full metrics, including every individual error
<code>out/...(redacted).docx</code>	The redacted document
<code>out/mapping.json</code>	real to fake lookup. Sensitive - don't ship this next to the redacted file.

Path	What it is
out/audit.jsonl	One line per replacement, for spot-checking
EVALUATION.md	Evaluation approach and results